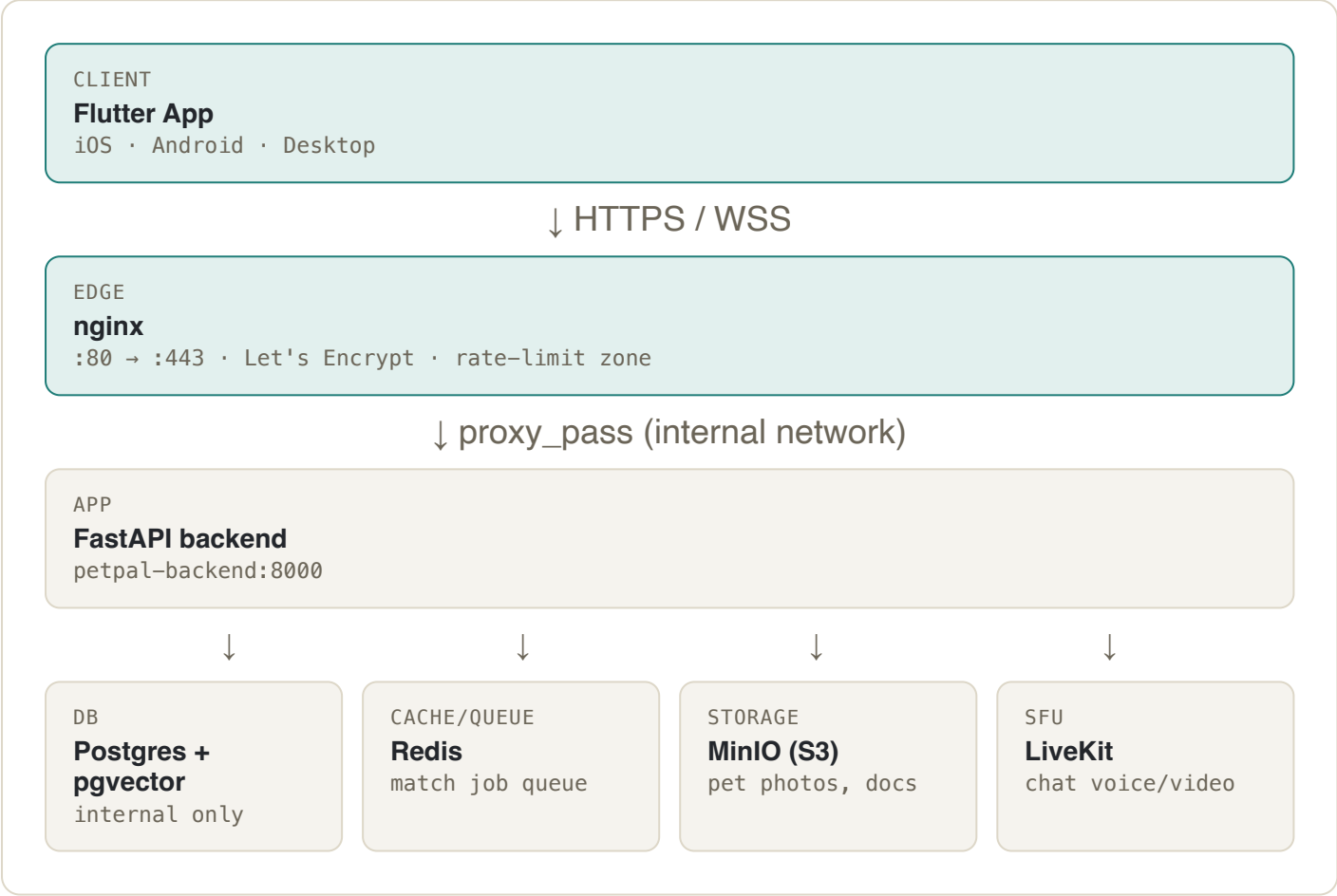


PetPal — Flow, Infrastructure & Test Coverage

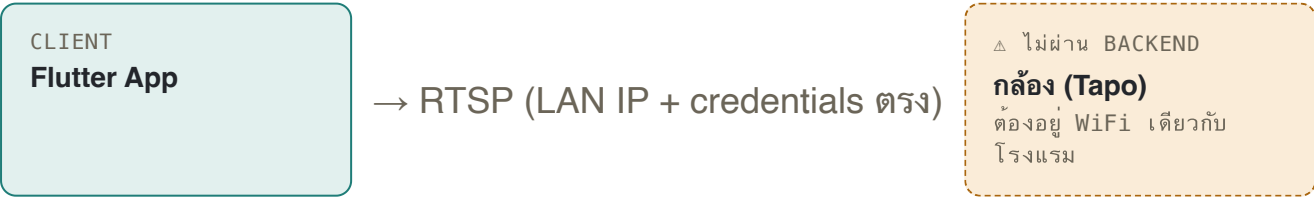
สรุปสถาปัตยกรรมระบบ, เส้นทางการทำงานหลัก (auth / booking-gated camera / AI match), และสถานะการทดสอบปัจจุบัน — จัดทำเพื่อให้ทีม Tech Support ที่เข้ามาช่วยดูแลระบบ เห็นภาพรวมทั้งหมดก่อนรับช่วงต่อ

สถาปัตยกรรมระบบปัจจุบัน

Backend รันบน VPS เดียว ผ่าน Docker Compose — nginx เป็นจุดเข้าเดียวที่เปิดสู่อินเทอร์เน็ต (TLS termination + reverse proxy) ส่วนบริการภายใน (DB, cache, storage, SFU) ไม่เปิด port ออกสู่นอก Docker network โดยตรง



เส้นทางกล้อง (แยกจากด้านบน) — ปัจจุบันยังไม่ผ่าน backend ในการส่งวิดีโอ:



SERVICE	บทบาท	EXPOSURE	หมายเหตุ
nginx	TLS + reverse proxy + rate limit	:80, :443 public	จุดเข้าเดียวจากอินเทอร์เน็ต
petpal-backend	FastAPI (business logic ทั้งหมด)	internal only	proxy ผ่าน nginx เท่านั้น

postgres (pgvector)	ข้อมูลหลัก + vector search สำหรับ AI match	internal only (prod)	เปิด :5433 เฉพาะ dev compose
redis	job queue ผลลัพธ์ AI match (async)	internal only	maxmemory 128mb, allkeys-lru
minio	เก็บไฟล์ (รูปสัตว์เลี้ยง, เอกสาร)	internal, proxy ผ่าน /minio/	console :9001 เข้าผ่าน SSH tunnel เท่านั้น
livekit	SFU สำหรับวิดีโอ/วอยซ์คอล ในแชท	:7880/:7881 + UDP 50000-50019	media flow ตรงถึง client ไม่ ผ่าน nginx

เส้นทางการทำงานหลัก 3 เส้นทาง

A. Authentication — รองรับ email/password และ social login

- ① **Client ส่ง credential**
Email+password ตรง หรือ ID token จาก Google / access token จาก Facebook

- ② **Backend ตรวจสอบ**
Google: verify ผ่าน `google.oauth2.id_token` เทียบ audience กับ `GOOGLE_CLIENT_ID` · Facebook: cross-check `debug_token` กับ app ID ของเราเอง (กัน token ของแอปอื่นมาสวมรอย)

- ③ **ออก JWT**
Access token อายุ 30 นาที + refresh token อายุ 14 วัน

- ⚠ **Rate limit ที่ `/auth`**
จำกัด 10 คำขอ/60 วิ ต่อ IP กัน brute-force — เก็บ state ใน memory เครื่องเดียว (ดูหัวข้อ 04)

B. Booking-gated Camera Access — ลูกค้าดูกล้องได้เฉพาะโรงแรมที่จองอยู่จริง

- ① **Client ขอ `GET /cameras/{id}/stream`**
ส่ง JWT มาด้วยตามปกติ

- ② **ตรวจสอบสิทธิ์ตาม role**
Admin/Technician ผ่านทันที · Customer ต้องมี booking ที่ `hotel_id` เดียวกับกล้อง และสถานะไม่ใช่ `cancelled`

- ③ **ถอดรหัสผ่านกล้อง**
Fernet-encrypted password ที่เก็บใน DB ถูก decrypt ฝั่ง server

- ⚠ **ส่ง URL กลับไปยัง client**
`rtsp://user:pass@ip:port/path` — credential หลุดไปถึง client ตรงๆ ยังไม่ผ่าน proxy/token แบบ short-lived (ดูหัวข้อ 04)

C. AI Smart Match — ค้นหาโรงแรมด้วยข้อความอิสระ + AI enrichment แบบ async

- ① **Vector search ทันท่วงที**
แปลงข้อความเป็น embedding (multilingual, รองรับไทย) ค้นด้วย pgvector ได้ผลลัพธ์กลับทันที

- ② **ส่งผลลัพธ์ local ก่อน**
ตอบ client ทันทีพร้อม `isFallback: true` — ไม่ต้องรอ AI

- ③ **Enrich ด้วย Gemini แบบ background**
Async task จำกัด concurrency (semaphore 3), timeout 8 วิ, เขียนผลลง Redis (`match_job:<id>`, TTL 5 นาที)

- ④ **Client poll ผลลัพธ์**
`GET /api/match/status/{job_id}` จนกว่าจะ `ready` หรือ fallback ต่อ

สถานะการทดสอบปัจจุบัน

รันด้วย pytest (backend) และ flutter test (client) — ยังเป็น manual run, ไม่มี CI

ชั้น	FRAMEWORK	ไฟล์	ครอบคลุม	สถานะ
Unit — backend	pytest	~14	camera crypto, rate limiter, cache, oauth verify, queue worker	• มีอยู่
Integration — API	pytest + httpx	6	API endpoints คุยกับ DB จริง (test_api, test_camera_probe, test_match_service ฯลฯ)	• มีอยู่
Widget — client	flutter_test	2	widget_test, match_api_service_test	• มีอยู่
Flow — client	flutter_test	2	hotel_partner_flow_test, semantic_match_test	• มีอยู่
CI automation	GitHub Actions	0	รันเทสอัตโนมัติตอน push/PR	• ยังไม่มี
Coverage %	pytest-cov	—	ตัวเลข coverage วัดจริง	• ยังไม่มี
Load / Performance	—	—	concurrent camera view / video call	• ยังไม่ต้องทำตอนนี้
Manual QA — real hardware	—	—	ทดสอบกับกล้องจริง, อุปกรณ์จริง (จับบั๊ก flutter_vlc_player ได้จากตรงนี้)	• ทำแล้ว ต่อเนื่อง

สิ่งที่ทีม Tech Support ควรรู้ก่อนรับช่วงต่อ

แจ้งตรงๆ ไว้ก่อน เพื่อให้ทีมที่มาซัพพอร์ตวางแผนช่วยแก้ต่อได้ถูกจุด ไม่ใช่ต้องมาไล่เจอเอง

🔴 Gemini model deprecated

backend/app/gemini_client.py:11 ตั้งค่า MODEL_NAME = "gemini-2.0-flash" ซึ่งถูกยกเลิกไปแล้วตั้งแต่ 1 มิ.ย. 2026 — ฟีเจอร์ AI Smart Match enrichment น่าจะเรียก API แล้ว fail อยู่ตอนนี้ ต้องเปลี่ยนเป็น gemini-2.5-flash-lite (ราคาเท่าเดิม)

🟡 RTSP credential หลุดถึง client

_build_stream_url ใน cameras.py ส่ง rtsp://user:pass@ip:port ตรงให้แอป — ยังไม่มี edge-server proxy หรือ short-lived token ตามสถาปัตยกรรมเป้าหมาย

🟡 กล้องดูได้เฉพาะใน WiFi โรงแรม

RTSP URL เป็น LAN IP — ลูกค้าที่ไม่ได้อยู่ในโรงแรมเชื่อมต่อไม่ได้เลย ต้องมี edge/Mini Hub ต่อโรงแรมถึงจะแก้ได้จริง (ไม่ใช่แค่เพิ่ม bandwidth)

🟡 ไม่มี CI

เทสที่มีอยู่ (~24 ไฟล์รวมสองฝั่ง) ไม่ได้รันอัตโนมัติเลยตอน push/PR — ต้องรัน pytest / flutter test มือทุกครั้ง

🟡 Rate limiter เป็น in-memory เดี่ยว

core/rate_limit.py เก็บ state ในหน่วยความจำเครื่องเดียว — ใช้ได้ตราบใดที่ backend มี instance เดี่ยว ถ้าจะ scale แนวนอนต้องย้ายไป Redis ก่อน (Redis มีอยู่แล้วในระบบ)